Domain 1 Master Cheat Sheet - SDLC Automation (22% of DOP-C02)

One place for the facts that win Domain 1 questions: what each service does, the numbers to remember, the deployment strategies, and a **keyword list** that maps the words in a question to the answer. Read Part 1 first, then use the keyword list (Part 10) as a last-day review.

Part 1 - How Domain 1 questions work

The four task statements

Task	What the exam asks you to do
1.1 Implement CI/CD pipelines	Source, build, test, deploy stages; triggers; cross-account and cross-Region pipelines; approvals; secrets in pipelines
1.2 Integrate automated testing	Which test runs where; failing the pipeline on bad results; reports; security scans
1.3 Build and manage artifacts	CodeArtifact, ECR, S3 artifacts, AMIs (Image Builder), versioning, encryption, retention
1.4 Deployment strategies	Instance (EC2/ASG, Beanstalk), container (ECS, EKS) and serverless (Lambda) deployments: rolling, blue/green, canary, linear, rollback

How to pick the right answer

1. **Find the requirement words:** least operational overhead, minimal downtime, fastest rollback, no extra cost, most secure, without code changes.
2. **Prefer managed and built-in features** over custom Lambda scripts or self-managed servers.
3. **Prefer automatic rollback** (alarms, circuit breaker, hooks) over manual steps.
4. **Prefer least privilege and no long-term keys** (roles, OIDC, connections, Secrets Manager).
5. **Eliminate answers with a wrong fact** (for example CodeDeploy for EKS, or weights on $LATEST).

Part 2 - Source: CodeCommit, CodeConnections, Git providers

Topic	Remember
CodeCommit	Managed Git. Encrypted with KMS. Access with IAM (HTTPS Git credentials, SSH keys, or **git-remote-codecommit** with roles/SSO). Still in exam scope
Branch protection	**IAM deny policy** on `GitPush`, `PutFile`, `MergePullRequest*`, `DeleteBranch` with condition `codecommit:References` for `refs/heads/main`
Approval rules	**Approval rule templates** attached to repositories: required approvers and pool (for example 2 from the leads role)
Events	**Notification rules** (to SNS / Amazon Q Developer in chat) for PRs and comments. **Triggers** (SNS or Lambda) on push / branch events. **EventBridge** for automation (start pipeline, run CodeBuild on PR)
Cross-account access	Role in the repository account trusted by the other account; users assume it (git-remote-codecommit)
CodeConnections (formerly CodeStar Connections)	Links **GitHub, GitHub Enterprise, GitLab, Bitbucket** to CodePipeline / CodeBuild. Created **PENDING**, completed by a handshake in the console. Pipeline role needs `codeconnections:UseConnection` (or `codestar-connections:UseConnection`). Replaces GitHub v1 OAuth tokens (deprecated)
Self-hosted GitHub Enterprise / GitLab	Needs a **host** resource with **VPC** settings if in a private network
GitHub Actions to AWS	**OIDC identity provider** in IAM + role trust on `token.actions.githubusercontent.com` with `sub` condition for the repo/branch. No access keys

Part 3 - Build: CodeBuild

Topic	Remember
buildspec.yml	Phases **install, pre_build, build, post_build**; `finally` blocks; `on-failure: ABORT` or `CONTINUE`; `artifacts`, `cache`, `reports` sections. Default location: repo root (override per project)
Failure rule	Any **non-zero exit code** fails the build. An always-true fallback on a command hides failures
Environment variables	Plain, `parameter-store`, `secrets-manager` (fetched at runtime, masked). Exported variables become **pipeline output variables**
Caching	**S3 cache** (shared across hosts) or **local cache**: Docker layer, source, custom
VPC builds	Needed to reach private resources (RDS, private APIs, private EKS). Needs **NAT** or endpoints for internet/AWS APIs
Docker builds	**Privileged mode** required for building images
Reports	**Test reports** (JUnit, NUnit, TestNG, Cucumber, TRX) and **coverage** (JaCoCo, Cobertura, Clover, SimpleCov, lcov). Kept **30 days**
Speed	**Batch builds** (matrix / graph / list), **parallel test splitting**, larger compute, caching, **reserved capacity fleets**
Timeout	Build timeout **5 min to 36 hours** (default 60 minutes); queue timeout separate
Local testing	**CodeBuild local agent**
Triggers	Webhooks from GitHub / Bitbucket / GitLab (filter groups by event, branch, file path), schedules, EventBridge
Logs	CloudWatch Logs and/or S3

Part 4 - Artifacts: CodeArtifact, ECR, S3

CodeArtifact

- **Domain** (shared storage, KMS key, one per org usually) contains **repositories**.
- **Upstream repositories** chain lookups; **external connection** (one per repo) to public registries: npmjs, PyPI, Maven Central, NuGet and others.
- Formats: **npm, PyPI, Maven/Gradle, NuGet, Swift, Ruby, Cargo, generic** and more.
- Auth: `aws codeartifact login` or `get-authorization-token` (token valid **12 hours by default**, max 12h; min 15 min).
- **Domain policy** and **repository policy** for cross-account access.
- **Package origin controls** (allow publish / allow upstream) block **dependency confusion** attacks.
- Private builds without internet: **VPC endpoints** `codeartifact.api`, `codeartifact.repositories` and **S3 gateway**.
- **EventBridge** events on package version changes (trigger pipelines when a new package version is published).

ECR

- **Lifecycle policies:** expire images by **count** or **age** (`sinceImagePushed`), tag status tagged / untagged / any, tag prefixes or patterns; rule **priority** (lower first).
- **Tag immutability** stops tag overwrites (with exclusion filters).
- **Scanning: basic** (on push or manual, OS packages) vs **enhanced = Amazon Inspector** (continuous, **OS + language packages**). Results in EventBridge.
- **Replication: cross-Region** and **cross-account** (registry setting, destination registry permission), filters by repository prefix. **Pull through cache** for public registries (Docker Hub needs credentials in Secrets Manager).
- **Repository policy** for cross-account pull; ECS/EKS pull with **task execution role / node role**.
- Encryption AES-256 default or **KMS**.

S3 pipeline artifacts

- CodePipeline artifact store = **S3 bucket per Region**, encrypted with **KMS**. **Cross-account** deploys need a **customer managed KMS key** (the default AWS-managed key can't be shared) plus bucket policy for the other account.
- **S3 Object Lock / versioning** for tamper-proof release artifacts.

AMIs

- **EC2 Image Builder** (see Part 9). Golden AMIs, immutable infrastructure.

Part 5 - Orchestration: CodePipeline

Topic	Remember
Structure	**Stages** with **actions**: Source, Build, Test, Deploy, Approval, Invoke. Same **runOrder** = parallel; any failure fails the stage
V1 vs V2	V2: **triggers with filters** (branches, file paths, tags, PRs), **pipeline-level variables**, **stage conditions/rules**, **execution modes**, retry/rollback on failure. Cost per action-minute
Execution modes (V2)	**SUPERSEDED** (default, newer run replaces older waiting), **QUEUED** (one at a time in order), **PARALLEL** (independent runs)
Stage conditions	**beforeEntry / onSuccess / onFailure** with rules: **CloudWatch alarm**, Lambda, VariableCheck, deployment window, commands
On failure	**Rollback** to last successful execution, or **retry** failed actions
Variables	Action **output variables** with **namespace**: `#{Namespace.Var}`; pipeline variables `#{variables.name}`
Manual approval	SNS notification, review URL, comments; times out after **7 days**
Lambda invoke action	Function must call **PutJobSuccessResult / PutJobFailureResult**
Change detection	**EventBridge** (CodeCommit, S3 with CloudTrail / EventBridge notifications, ECR push) or **CodeConnections** webhooks. Avoid polling
Cross-Region	Actions in other Regions need an **artifact bucket (and KMS key) in each Region**
Cross-account	Customer managed **KMS key** shared with target account; **artifact bucket policy**; **cross-account role** in the target account the pipeline action assumes (`roleArn` on the action)
Monitoring	EventBridge **pipeline / stage / action state change** events, notification rules

Part 6 - Deploy: CodeDeploy

Topic	Remember
Platforms	**EC2/on-premises, Lambda, ECS**. **Not EKS**
EC2 deployments	**In-place** or **blue/green** (new ASG / instances behind ELB). Agent required. On-prem instances registered with IAM user or **role (IAM session / Roles Anywhere)**
EC2 hooks order	ApplicationStop, DownloadBundle, **BeforeInstall**, Install, **AfterInstall**, **ApplicationStart**, **ValidateService** (plus BeforeBlockTraffic / AfterBlockTraffic / BeforeAllowTraffic / AfterAllowTraffic with a load balancer)
EC2 configs	**AllAtOnce, HalfAtATime, OneAtATime** or custom minimum healthy hosts (count or %)
Lambda	Hooks **BeforeAllowTraffic / AfterAllowTraffic**; configs **Canary10Percent5/10/15/30Minutes, Linear10PercentEvery1/2/3/10Minutes, AllAtOnce**
ECS	Blue/green with **two target groups**, prod + optional **test listener**; hooks BeforeInstall, AfterInstall, **AfterAllowTestTraffic**, BeforeAllowTraffic, AfterAllowTraffic; canary/linear need **ALB** (NLB = AllAtOnce); original task set wait up to **2 days**
Rollback	**Automatic rollback** on deployment failure or **CloudWatch alarm**; rollback = **redeploy last good revision** (new deployment ID)
Common failures	Agent not running / no instance profile S3 access; scripts failing (logs in `/opt/codedeploy-agent/deployment-root/...`); ApplicationStop fails because old revision scripts are broken (use `--ignore-application-stop-failures`); file exists errors (`file_exists_behavior`: DISALLOW, OVERWRITE, RETAIN)
Hook timeout	Default **3600 s (1 hour)** per lifecycle event script

Part 7 - Deployment strategies (every service)

Patterns

Pattern	Downtime	Capacity	Mixed versions	Rollback	Cost
All at once / in place	Yes	Drops	No	Redeploy	Lowest
Rolling	No	Reduced	Yes	Redeploy	Low
Rolling with additional batch	No	Full	Yes	Redeploy	Small extra
Immutable	No	Full (double briefly)	Briefly	Terminate new	High
Blue/green	No	Double	No (switch)	**Switch back (fastest)**	Highest
Canary	No	Extra	Yes (small %)	Automatic	Medium
Linear	No	Extra	Yes	Automatic	Medium
Feature flags (AppConfig)	No	Same	Code paths	Toggle off	Low

By service

Service	Rolling	Blue/green	Canary / linear	Automatic rollback
Elastic Beanstalk	Rolling, rolling with additional batch	**Swap environment URLs** (DNS)	**Traffic splitting** (ALB, %, evaluation time)	Immutable / traffic splitting on health
EC2 / ASG	**Instance refresh**, CloudFormation `AutoScalingRollingUpdate`	CodeDeploy blue/green, `AutoScalingReplacingUpdate`	ALB weighted target groups	Instance refresh alarms, CodeDeploy alarms
Lambda	n/a	Alias switch	**SAM DeploymentPreference / CodeDeploy** (weighted alias)	Alarms + PreTraffic / PostTraffic hooks
ECS	`minimumHealthyPercent` / `maximumPercent`	**CodeDeploy** (or ECS built-in blue/green)	CodeDeploy ECSCanary / ECSLinear	**Circuit breaker** + alarms (rolling); CodeDeploy alarms
EKS	Deployment `maxSurge` / `maxUnavailable`	Service selector switch, new cluster + Route 53	**Argo Rollouts / Flagger**, ALB Ingress weights	Rollouts analysis, `helm --atomic`
API Gateway	n/a	Stage switch	**Canary release** on a stage	Manual promote / delete canary
CloudFront	n/a	n/a	**Continuous deployment** (staging distribution, % or header)	Promote or discard

Beanstalk policies in one line

- All at once (fast, downtime), Rolling (reduced capacity), Rolling with additional batch (full capacity), **Immutable** (temporary ASG, safest), **Traffic splitting** (canary). **Single-instance** = all at once or immutable only. **LB type**

or platform branch change = new environment + swap URLs. `.ebextensions` (`container_commands` with `leader_only`). Worker tier = **SQS + sqsd** + `cron.yaml`.

Lambda in one line

- **Versions immutable** (code + config incl. env vars), **$LATEST mutable**; **alias** = pointer (stable ARN, per-alias permissions); **weighted alias** = 2 published versions, same role and DLQ; SAM **AutoPublishAlias + DeploymentPreference** (Type, Alarms, Hooks named **CodeDeployHook**_*); provisioned concurrency on **alias/version**, never $LATEST.

ECS in one line

- **Execution role** = pull/logs/secrets at start; **task role** = app calls; **rolling** 100/200 default, **100/100 stuck**, **50/100** no extra capacity; **circuit breaker** (rolling only) + **alarms**; **CodeDeploy blue/green** (test listener, `AfterAllowTestTraffic`); pipeline files `imagedefinitions.json` (rolling) vs `imageDetail.json` **+** `taskdef.json` **+** `appspec.yaml` (blue/green); secrets read at start, **force new deployment** after rotation.

EKS in one line

- Pipeline needs `eks:DescribeCluster` **+ access entry** (Unauthorized = not mapped); pods use **Pod Identity / IRSA**; **readinessProbe** for zero-error rollouts; `kubectl rollout status / undo`; **no CodeDeploy for EKS**; GitOps (Argo CD / Flux) = no creds in CI; **audit logs** off by default.

Part 8 - Testing and security in pipelines

Stage	Tests and tools
Pre-commit / PR	Lint, unit, **git-secrets**
Build	**Unit**, SAST (Amazon Q Developer code review, CodeGuru Reviewer, Inspector code scanning), **SCA** (Inspector SBOM), secrets scan, **IaC** (cfn-lint, **cfn-guard**, cfn-nag, Checkov), **image scan**
Test env	**Integration** (ephemeral stacks + output variables), API/contract, E2E, **Device Farm**
Staging	**Load** (Distributed Load Testing on AWS, k6, JMeter with thresholds), **DAST** (OWASP ZAP), **FIS** (stop conditions = alarms), UAT + **manual approval**
Production	Canary / linear with **alarms**, **CloudWatch Synthetics canaries**, deployment hooks

- **CodeGuru Reviewer** = code review recommendations, **secrets detection** (hard-coded credentials), on PRs or full repository scans. **CodeGuru Profiler** = runtime CPU/latency hotspots and cost of code (agent in the app). Much of this is moving to **Amazon Q Developer** and **Inspector**.
- Coverage threshold must be enforced by the **test tool exit code**.

Part 9 - AMI pipelines: EC2 Image Builder

- **Component** (AWSTOE YAML: build, **validate** on build instance, **test** on new instance from new AMI; immutable versions) + **recipe** (parent with `x.x.x`) + **infrastructure config** (instance profile **SSM core + EC2InstanceProfileForImageBuilder**, route to SSM/S3, `terminateInstanceOnFailure`) + **distribution** (Regions, accounts/OUs, KMS, **launch template**, **SSM parameter**) = **pipeline** (cron, **only on dependency updates**, Inspector scanning, workflows with **WaitForAction**).
- **Lifecycle policies** clean old AMIs/snapshots. Rollout: **EventBridge AVAILABLE -> Lambda -> launch template -> ASG instance refresh**. Cross-account encrypted AMIs need a **customer managed KMS key**.

Part 10 - Domain 1 keyword list (question words to answer)

Source and access

If the question says...	Think...
Prevent direct pushes to main / only leads can merge	IAM deny on GitPush with `codecommit:References` + approval rule templates
Notify Slack/Teams on PR comments	Notification rule to Amazon Q Developer in chat applications (SNS)
Run a build when a PR is created	EventBridge rule (CodeCommit PR event) or webhook filter to CodeBuild
GitHub / Bitbucket / GitLab source in CodePipeline	CodeConnections + `UseConnection` permission
Connection stuck in PENDING	Complete the handshake in the console
GitHub Actions needs AWS access without keys	IAM OIDC provider + role with `sub` condition
On-premises servers need AWS credentials	IAM Roles Anywhere (X.509 certificates)
Developers in another account need repo access	Cross-account role + git-remote-codecommit

Build and artifacts

If the question says...	Think...
Build needs a database password	buildspec `env: secrets-manager:`
Build must reach a private RDS / private API	CodeBuild in VPC (+ NAT / endpoints)
Docker build fails with permission error in CodeBuild	Privileged mode
Slow dependency downloads	CodeBuild cache (S3 or local), CodeArtifact
Test on several runtime versions at once	Batch build (matrix)
Private npm/PyPI/Maven repository, public upstream	CodeArtifact + upstream + external connection
Dependency confusion / malicious public package	CodeArtifact package origin controls
Packages shared across accounts	CodeArtifact domain/repository policy
Old container images cost money	ECR lifecycle policy
Tags overwritten / deployed wrong image	ECR tag immutability + unique tags / digests
Language package CVEs in images	ECR enhanced scanning (Inspector)
Same images in other Regions/accounts	ECR replication
Avoid Docker Hub rate limits	ECR pull through cache
Tamper-proof release artifacts	S3 Object Lock / versioning

Pipelines

If the question says...	Think...
Only run pipeline for changes in a folder or branch	CodePipeline V2 trigger filters (file paths, branches)
Newer commits shouldn't cancel a running release	Execution mode QUEUED (or PARALLEL)
Pass a value from one action to the next	Output variables `#{Namespace.Var}`
Block a stage while an alarm is firing	V2 stage condition, CloudWatch alarm rule
Automatic rollback when a stage fails	V2 onFailure rollback
Human sign-off before production	Manual approval (7 days, SNS)

If the question says...	Think...
Custom check step hangs	Lambda must call PutJobSuccessResult/FailureResult
Deploy to another Region	Artifact bucket + KMS key in that Region
Deploy to another account	Customer managed KMS key + bucket policy + cross-account role
Start pipeline when S3 object uploaded / image pushed	EventBridge rule
Notify on pipeline failure	EventBridge pipeline state change or notification rule

Deployments

If the question says...	Think...
Fastest rollback, no mixed versions	Blue/green
Test with small % of real users, auto rollback	Canary with alarms
Equal steps over time	Linear
Full capacity, lowest extra cost (Beanstalk)	Rolling with additional batch
Failed deploy must not touch running instances (Beanstalk)	Immutable
Beanstalk 10% for 15 minutes	Traffic splitting
Change LB type / platform branch (Beanstalk)	New environment + swap URLs
Migration runs once (Beanstalk)	`container_commands` + `leader_only`
Long jobs time out in web tier	Beanstalk worker environment (SQS)
Scheduled job runs on every instance	Worker `cron.yaml`
Beanstalk database deleted with environment	RDS outside Beanstalk
Lambda gradual deploy + auto rollback, least code	SAM AutoPublishAlias + DeploymentPreference
Test Lambda version before traffic	PreTraffic hook
Hook AccessDenied	Name `CodeDeployHook_*`
API Gateway 500 after alias switch	add-permission --qualifier alias
Fastest Lambda rollback	update-alias to previous version
ECS rolling deploy auto rollback	Deployment circuit breaker (+ alarms)
ECS test port before production	CodeDeploy blue/green, test listener, AfterAllowTestTraffic
ECS app AccessDenied	Task role
ECS can't pull image / read secret	Execution role (+ route to ECR/SSM)
ECS deploy stuck at desired 1, 100/100	Raise maximumPercent
ECS rolling pipeline file	imagedefinitions.json
ECS blue/green pipeline files	imageDetail.json + taskdef.json + appspec.yaml
CodeDeploy EC2 smoke test	ValidateService hook
CodeDeploy ApplicationStop keeps failing	Old scripts broken: ignore-application-stop-failures
EKS kubectl Unauthorized	Access entry (or aws-auth)
Canary on EKS	Argo Rollouts / Flagger (not CodeDeploy)
New AMI to ASG safely	Instance refresh (min healthy, checkpoints, alarm rollback)
Feature on/off without deploying	AppConfig feature flags
CloudFront gradual change	Continuous deployment (staging distribution)
API canary	API Gateway stage canary

Testing and AMIs

If the question says...	Think...
Fail below coverage threshold	Test tool exits non-zero
Keep test results 1 year	Export reports to S3
Integration tests need the new stack URL	Output variables
Template must follow company rules	cfn-guard / Guard Hooks
Secrets committed to Git	git-secrets pre-commit + pipeline scan, CodeGuru Reviewer secrets detector
Find CPU hotspots in production code	CodeGuru Profiler
Load test before release	Staging + thresholds (Distributed Load Testing on AWS)
Stop chaos test on errors	FIS stop conditions
Scripted checks in production	CloudWatch Synthetics
Golden AMI, least effort	EC2 Image Builder
Image build only when base changed	DEPENDENCY_UPDATES_AVAILABLE condition
Stacks always use newest AMI	SSM parameter + `AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>`
Image build fails early, no logs	Instance profile SSM + network route

Part 11 - Numbers to remember

Item	Value
Domain 1 weight	**22%**
Manual approval timeout	**7 days**
CodeBuild report retention	**30 days**
CodeBuild timeout	5 minutes to **36 hours** (default 60 min)
CodeArtifact auth token	Default and max **12 hours**
CodeDeploy lifecycle hook script timeout	Default **3600 s**
CodeDeploy/SAM hook status	Must report within **1 hour**
ECS rolling defaults	**min 100% / max 200%**
ECS circuit breaker threshold	Half desired count, **min 3, max 200**
ECS blue/green termination wait	Up to **2,880 min (2 days)**
Lambda weighted alias	**2 versions** max
Lambda code storage	**75 GB** per Region (default)
Beanstalk application versions	**1,000** per Region (lifecycle policy)
Beanstalk command timeout	**600 s** default
Beanstalk source bundle	**500 MB** max
Beanstalk terminated environment restore	Within **6 weeks**
Fargate stopTimeout	Up to **120 s**
Fargate Spot warning	**2 minutes**
EKS upgrades	**One minor version** at a time
Canary configs (Lambda)	10% for **5, 10, 15, 30** minutes
Linear configs (Lambda)	10% every **1, 2, 3, 10** minutes

Part 12 - Classic wrong answers (distractors)

- **CodeDeploy for EKS** (not supported).
- **Weighted alias with $LATEST** (not allowed).
- **Provisioned concurrency on $LATEST** (not allowed).
- **IAM permissions alone for kubectl** (needs an access entry).
- **Task execution role for app permissions** (use the task role).
- **Node role for pod permissions** (use Pod Identity / IRSA).
- **Polling the source** instead of EventBridge or connections.
- **AWS-managed KMS key for cross-account artifacts** (use a customer managed key).
- **Custom Lambda to shift traffic** when SAM / CodeDeploy / circuit breaker does it built in.
- **Long-term access keys** in CI tools (use OIDC, roles, Roles Anywhere).
- **Secrets in buildspec, `.ebextensions`, task definition environment or AMIs**.
- **Load testing production on every commit**.
- **cfn-lint for custom policy rules** (use cfn-guard).
- **ECR basic scanning for npm/pip CVEs** (use enhanced).
- **Editing a published Lambda version, component version or task definition revision** (all immutable; create a new one).